

USN

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Second Semester MCA Degree Examination, June/July 2025

Object Oriented Programming using Java

Time: 3 hrs.

Max. Marks: 100

*Note: 1. Answer any FIVE full questions, choosing ONE full question from each module.
2. M: Marks, L: Bloom's level, C: Course outcomes.*

Module – 1				M	L	C
Q.1	a.	List and explain the features of java.		10	L2	CO1
	b.	Explain narrowing and widening type casting with example.		7	L2	CO1
	c.	Write short note on Garbage collector.		3	L1	CO1
OR						
Q.2	a.	Explain <i>this</i> keyword with example program.		6	L3	CO1
	b.	Describe object as parameter with example.		6	L2	CO1
	c.	Explain the following: i) Access specifier ii) command-line argument		8	L2	CO1
Module – 2						
Q.3	a.	What is inheritance? Explain single inheritance with suitable example program		7	L3	CO2
	b.	Write a note on final with inheritance		6	L1	CO2
	c.	Describe the role of the super keyword in Java. Provide a suitable Java program to demonstrate its functionality.		7	L2	CO2
OR						
Q.4	a.	What is a String in Java? List the different constructors of the String class can be used to create string objects. Explain any two.		6	L2	CO2
	b.	Explain how to modify a string by using following methods with syntax i) substring() ii) concat() iii) replace() iv) trim()		8	L2	CO2
	c.	Define generics in Java. Write a Java program to demonstrate how generics enhance type safety and eliminate the need for explicit type casting.		6	L2	CO2
Module – 3						
Q.5	a.	Define package. Explain how to create and import a user defined package in java with an example.		10	L3	CO3
	b.	Explain the concept of interfaces in Java. How do interfaces support multiple inheritance? Write a Java program to demonstrate the implementation of an interface.		10	L3	CO3
OR						
Q.6	a.	What is the use of multiple catch statement in Exception handling? Discuss with a java program		10	L2	CO3
	b.	Write a java program which uses throws keyword for handling Exception.		10	L3	CO3

Module – 4

Q.7	a.	Define a Thread. Explain the different ways of creating threads.	7	L2	CO4
	b.	What is the need of synchronization? Explain with an example how synchronization is implemented in java.	7	L3	CO4
	c.	Write short note on, i) isAlive() ii) join()	6	L1	CO4

OR

Q.8	a.	Write a program to create multiple threads in Java	8	L3	CO4
	b.	Explain the concept of thread priorities in Java. How do they affect thread execution?	6	L2	CO4
	c.	What do you understand by Byte Streams in Java? Describe their working and importance in handling binary data.	6	L1	CO4

Module – 5

Q.9	a.	Write a Java program to handle mouse events using the MouseListener interface.	10	L3	CO5
	b.	Explain the following methods of Graphics class with example i) drawRect() ii) drawOval()	10	L3	CO5

OR

Q.10	a.	Explain the fundamental components of a window in a graphical user interface	10	L2	CO5
	b.	Explain how to create and display a Frame window in Java AWT. Provide a sample program to demonstrate this.	10	L2	CO5

BWA55

Q.1 a. List and explain the features of Java.

Java is a high-level, object-oriented programming language designed for portability, robustness, and ease of development. Its principal features:

1. **Platform independence (WORA)** — Java source is compiled to bytecode (.class) executed by the JVM. The same bytecode runs on any platform with a compatible JVM. This decouples program logic from hardware/OS specifics.
2. **Object-oriented** — Java models real-world entities as objects. Key OO principles supported: encapsulation (data hiding via access modifiers), inheritance (reuse/extension), polymorphism (method overriding and interfaces), and abstraction (abstract classes/interfaces).
3. **Robust** — Strict compile-time and runtime type checking, exception handling, and absence of explicit memory pointers reduce runtime errors and enhance program stability.
4. **Secure** — Class loader, bytecode verifier, and security manager help restrict untrusted code; no pointer arithmetic prevents memory corruption.
5. **Automatic memory management (GC)** — Garbage Collector reclaims unreachable objects, preventing common memory leaks and double-free errors.
6. **Multithreaded** — Built-in support for threads and synchronization enables concurrent programming.
7. **Portable** — Standard class libraries and consistent behavior across JVMs make Java applications portable.
8. **High performance** — Just-In-Time (JIT) compilers and HotSpot optimizations improve runtime speed.
9. **Distributed** — Rich networking libraries (java.net) simplify client-server and distributed systems development.
10. **Dynamic & extensible** — Reflection and dynamic class loading allow runtime behavior that adapts to new classes.

Q.1 b. Explain narrowing and widening type casting with example.

Widening conversion (implicit) enlarges the data type (no data loss expected): byte → short → int → long → float → double. The compiler inserts conversion automatically.

```
int i = 100;
```

```
double d = i; // widening: automatic, d == 100.0
```

Narrowing conversion (explicit) converts from a larger to smaller data type; may lose information — programmer must force cast:

```
double x = 9.99;
```

```
int y = (int) x; // narrowing: y == 9 (fraction lost)
```

Reference types:

- **Upcasting** (subclass → superclass) is implicit and safe.
- **Downcasting** (superclass → subclass) requires explicit cast and may cause `ClassCastException` at runtime if the actual object isn't of the target type.

```
Animal a = new Dog();    // upcast, implicit
```

```
Dog d = (Dog) a;         // downcast, explicit
```

Q.1 c. Write short note on Garbage collector.

The Garbage Collector (GC) in Java automatically reclaims memory by removing objects that are no longer reachable from any live reference. GC runs on JVM-managed threads and uses various algorithms: generational collection (young/old generations), mark-and-sweep, and advanced collectors (G1, ZGC, Shenandoah). Developers should not depend on `System.gc()` to force collection; instead design code to minimize object churn and explicitly release non-memory resources (files, sockets) via `try-with-resources` and `AutoCloseable`.

Note: automatic memory reclaim, generational concept, resource cleanup best practices (`try-with-resources`), avoid finalizers.

Q.2 a. Explain this keyword with example program.

this refers to the current object instance inside instance methods and constructors. Uses:

- Distinguish instance variables from parameters (shadowing).
- Call another constructor: `this(...)`.
- Pass current instance as argument: `method(this)`.

Example:

```

public class Person {
    private String name;

    public Person(String name) {
        this.name = name; // 'this' distinguishes field from parameter
    }

    public void printName() {
        System.out.println("Name: " + this.name);
    }

    public static void main(String[] args) {
        Person p = new Person("Alice");
        p.printName();
    }
}

```

Q.2 b. Describe object as parameter with example.

Passing objects to methods passes the reference value (pass-by-value of reference). The method receives a reference to the same object and can modify its state.

```

class Student {
    String name;

    Student(String name) { this.name = name; }
}

```

```

public class Demo {
    static void display(Student s) {
        System.out.println("Student: " + s.name);
    }

    public static void main(String[] args) {

```

$$\}$$

Q.2 c i. Access specifier

Java access modifiers control visibility:

- `private` — accessible only in the declaring class.

Use them to enforce encapsulation and API control.

```
// File: AccessDemo.java
```

```
package demo;
```

```
class Base {
```

```
public int pubVar = 10;    // accessible everywhere
```

```
private int privVar = 20;    // accessible only within this class
```

```
protected int protVar = 30; // accessible in same package + subclasses
```

```
int defVar = 40;           // default (package-private), same package only
```

```
public void showBase() {
```

```
System.out.println("Base: public=" + pubVar +
```

```

", private=" + privVar +

```

```

", protected=" + protVar +

```

```
", default=" + defVar);
```

```
    }  
}
```

```
class SubClass extends Base {  
    public void showSub() {  
        System.out.println("Subclass: public=" + pubVar);  
        // System.out.println(privVar);  
        System.out.println("protected=" + protVar);  
        System.out.println("default=" + defVar);  
    }  
}
```

```
public class AccessDemo {  
    public static void main(String[] args) {  
        Base b = new Base();  
        b.showBase();  
        System.out.println("Main: public=" + b.pubVar);  
        // System.out.println(b.privVar);  
        System.out.println("Main: protected=" + b.protVar);  
        System.out.println("Main: default=" + b.defVar);  
  
        SubClass sc = new SubClass();  
        sc.showSub();  
    }  
}
```

Q.2 c ii. Command-line argument

Command-line arguments are provided to main via `String[] args`. They are strings; developers parse them into required types (e.g., `Integer.parseInt(args[0])`).

```
public class CmdDemo {  
  
    public static void main(String[] args) {  
  
        for (int i = 0; i < args.length; i++) {  
  
            System.out.println("Arg " + i + ": " + args[i]);  
  
        }  
  
    }  
  
}
```

Q.3 a. What is inheritance? Explain single inheritance with suitable example program.

Inheritance lets a class (subclass/derived) acquire fields and methods from another class (superclass/base), supporting code reuse and hierarchical classification.

Single inheritance means a subclass extends one superclass (Java permits single class inheritance).

```
class Animal {  
  
    public void eat() { System.out.println("Animal eats"); }  
  
}
```

```
class Dog extends Animal { // single inheritance  
  
    public void bark() { System.out.println("Dog barks"); }  
  
}
```

```
public class Test {  
  
    public static void main(String[] args) {  
  
        Dog d = new Dog();  
  
    }  
  
}
```

```
d.eat(); // inherited method

d.bark();

}

}
```

Q.3 b. Write a note on final with inheritance.

final usage:

- final class — cannot be subclassed (e.g., public final class Math {}).

```
final class Utility {

    static void printMessage() {

        System.out.println("Utility method.");

    }

}
```

```
// class MyUtility extends Utility { }
```

final method — cannot be overridden in subclasses.

```
class Parent {

    final void show() {

        System.out.println("This is a final method in Parent.");

    }

}
```

```
class Child extends Parent {

    // void show() {

    //     System.out.println("Trying to override");

    // }
```

```
}
```

- final variable — value cannot change after initialization (constant); with static final becomes compile-time constant.

```
class FinalVarDemo {
```

```
    final int MAX_VALUE = 100; // must be initialized once
```

```
    void display() {
```

```
        System.out.println("MAX_VALUE = " + MAX_VALUE);
```

```
        // MAX_VALUE = 200;
```

```
    }
```

```
}
```

Example:

```
class Demo {
```

```
    final int constant; // blank final variable
```

```
    Demo(int value) {
```

```
        constant = value;
```

```
    }
```

```
    final void display() {
```

```
        System.out.println("Constant = " + constant);
```

```
    }
```

```
}
```

```
final class FinalClass {
```

```
    void sayHi() {
```

```
        System.out.println("Hi from final class!");
    }
}
```

```
public class FinalKeywordExample {
    public static void main(String[] args) {
        Demo d = new Demo(50);
        d.display();

        FinalClass fc = new FinalClass();
        fc.sayHi();
    }
}
```

Reason: Use final to enforce immutability or prevent modification of behavior, improving security and design guarantees.

Q.3 c. Describe the role of the super keyword in Java. Provide a suitable Java program to demonstrate its functionality.

super refers to the immediate parent (superclass) and is used to:

- Access superclass member variables when shadowed.
- Call superclass methods (super.method()).
- Call superclass constructor (super(...)) — must be first statement in constructor.

Example:

```
class Person {
    String name;

    Person(String name) { this.name = name; }

    void show() { System.out.println("Person: " + name); }
```

```
}
```

```
class Student extends Person {  
    int roll;  
    Student(String name, int roll) {  
        super(name); // call parent constructor  
        this.roll = roll;  
    }  
    void show() {  
        super.show(); // call parent method  
        System.out.println("Roll: " + roll);  
    }  
}
```

```
public class Demo {  
    public static void main(String[] args) {  
        Student s = new Student("Amy", 12);  
        s.show();  
    }  
}
```

Q.4 a. What is a String in Java? List the different constructors of the String class can be used to create string objects. Explain any two.

String in Java is an immutable sequence of characters (stored internally as char[] or in newer versions as byte[] with encoding). Key constructors:

- String() — creates empty string "".
- String(String original) — creates a new string from existing string.
- String(char[] value) — from character array.
- String(char[] value, int offset, int count) — subarray.
- String(byte[] bytes) — from byte array using platform default charset.
- String(byte[] bytes, Charset cs) — from bytes with charset.
- String(StringBuffer buffer) / String(StringBuilder builder) — from buffers/builders.

Explain for any 2:

- String(String original) — copies the original string reference content to new String (strings are immutable so often same internal char array is shared internally).
- String(char[] value) — useful to construct a string from characters produced or read.

Example:

```
public class StringConstructorDemo {  
    public static void main(String[] args) {  
        // 1. Using String(String original)  
  
        String s1 = "Hello Java";  
  
        String s2 = new String(s1); // creates a new String from existing String  
  
        System.out.println("Original String (s1): " + s1);  
  
        System.out.println("Copied String (s2): " + s2);  
  
        // 2. Using String(char[] value)
```

```

char[] letters = { 'J', 'A', 'V', 'A' };

String s3 = new String(letters); // creates String from char array

System.out.println("String from char array (s3): " + s3);

// 3. Using String(char[] value, int offset, int count)

String s4 = new String(letters, 1, 3); // takes subarray [A,V,A]

System.out.println("Substring from char array (s4): " + s4);

}

}

```

Q.4 b. Explain how to modify a string by using following methods with syntax: substring(), concat(), replace(), trim().

String methods return new Strings (immutability):

substring(int beginIndex) / substring(int beginIndex, int endIndex) — extracts parts.

```
String s = "HelloWorld";
```

```
String w = s.substring(5); // "World"
```

concat(String str) — joins two strings (same as + operator).

```
"Hello".concat(" Java"); // "Hello Java"
```

replace(CharSequence target, CharSequence replacement) — replace occurrences.

```
"banana".replace("a", "o"); // "bonono"
```

trim() — removes leading and trailing whitespace.

```
" hi ".trim(); // "hi"
```

Example:

```

public class StringMethodsDemo {

    public static void main(String[] args) {

        String s = " HelloWorld ";
    }
}

```

```

// 1. substring()

String part1 = s.substring(2, 7); // from index 2 to 6 -> "Hello"

System.out.println("Substring (2,7): " + part1);


// 2. concat()

String joined = part1.concat(" Java");

System.out.println("After concat: " + joined);


// 3. replace()

String replaced = joined.replace("Java", "Programming");

System.out.println("After replace: " + replaced);


// 4. trim()

String trimmed = s.trim(); // removes spaces from start and end

System.out.println("After trim: " + trimmed + "");

}

}

```

Q.4 c. Define generics in Java. Write a Java program to demonstrate how generics enhance type safety and eliminate the need for explicit type casting.

Generics enable parameterized types so classes/methods operate on typed data without casts. They provide compile-time type safety and eliminate many runtime `ClassCastException`s.

Example:

```

class Box<T> {

    private T value;

    public void set(T value) { this.value = value; }

    public T get() { return value; }
}

```

```
}
```

```
public class GenDemo {  
    public static void main(String[] args) {  
        Box<String> sb = new Box<>();  
        sb.set("Hello");  
        String s = sb.get(); // no cast needed  
  
        Box<Integer> ib = new Box<>();  
        ib.set(10);  
        Integer i = ib.get();  
    }  
}
```

Q.5 a. Define package. Explain how to create and import a user-defined package in Java with an example.

A **package** groups related classes/interfaces into a namespace to avoid name conflicts and to control visibility.

Create package: directory structure maps to package name.

```
package mypack;  
  
public class Greeting {  
    public static void sayHello() { System.out.println("Hello from mypack"); }  
}  
  
src/app/Test.java:  
  
package app;  
  
import mypack.Greeting;
```

```
public class Test {  
    public static void main(String[] args) {  
        Greeting.sayHello();  
    }  
}
```

Compile & run from src:

```
javac mypack/Greeting.java app/Test.java
```

```
java app.Test
```

Q.5 b. Explain the concept of interfaces in Java. How do interfaces support multiple inheritance? Write a Java program to demonstrate the implementation of an interface.

An interface declares method signatures and (since Java 8) default and static methods. A class can implement multiple interfaces, allowing multiple inheritance of type (not implementation) — avoids diamond-problem of multiple class inheritance.

Example:

```
interface Flyer { void fly(); }
```

```
interface Swimmer { void swim(); }
```

```
class Duck implements Flyer, Swimmer {
```

```
    public void fly() { System.out.println("Duck flies"); }
```

```
    public void swim() { System.out.println("Duck swims"); }
```

```
}
```

```
public class IfDemo {
```

```
    public static void main(String[] args) {
```

```
        Duck d = new Duck();
```

```
        d.fly(); d.swim();
```

```
}  
  
}
```

Q.6 a. What is the use of multiple catch statement in Exception handling? Discuss with a Java program.

Multiple catch blocks allow handling different exception types distinctly. Since Java 7 you can use multi-catch with.

```
import java.io.*;
```

```
public class MultiCatch {
```

```
    public static void main(String[] args) {
```

```
        try {
```

```
            int[] arr = new int[2];
```

```
            System.out.println(arr[5]); // may throw ArrayIndexOutOfBoundsException
```

```
            BufferedReader br = new BufferedReader(new FileReader("nofile.txt"));
```

```
        } catch (ArrayIndexOutOfBoundsException e) {
```

```
            System.out.println("Array index error: " + e.getMessage());
```

```
        } catch (FileNotFoundException e) {
```

```
            System.out.println("File not found: " + e.getMessage());
```

```
        } catch (IOException e) {
```

```
            System.out.println("IO error: " + e.getMessage());
```

```
        }
```

```
    }
```

```
}
```

Example:

```
try {
```

```
    // code that may throw IOException or SQLException
```

```
} catch (IOException | SQLException ex) {  
    System.out.println("IO or SQL exception: " + ex.getMessage());  
}
```

Q.6 b. Write a Java program which uses throws keyword for handling Exception.

throws declares checked exceptions that a method may pass to its caller.

```
import java.io.*;
```

```
public class ThrowsExample {  
    static void readFile(String path) throws IOException {  
        try (BufferedReader br = new BufferedReader(new FileReader(path))) {  
            System.out.println(br.readLine());  
        }  
    }  
  
    public static void main(String[] args) {  
        try {  
            readFile("input.txt");  
        } catch (IOException e) {  
            System.out.println("Handled: " + e.getMessage());  
        }  
    }  
}
```

Q.7 a. Define a Thread. Explain the different ways of creating threads.

A Thread is a unit of execution within a process. Java supports concurrency via threads.

Two primary ways to create threads:

1. Extend Thread class: override run(). Instantiate and call start() (which invokes run() on new thread).

Example:

```
class MyThread extends Thread {  
    public void run() { System.out.println("Thread running"); }  
}
```

```
MyThread t = new MyThread();
```

```
t.start();
```

Implement Runnable interface: implement run(); create Thread with new Thread(new RunnableImpl()), then start().

```
class MyTask implements Runnable {  
    public void run(){ System.out.println("Runnable running"); }  
}
```

```
Thread t = new Thread(new MyTask());
```

```
t.start();
```

Q.7 b. What is the need of synchronization? Explain with an example how synchronization is implemented in Java.

Synchronization prevents race conditions. Example: synchronized void increment() ensures mutual exclusion.

Without synchronization, multiple threads accessing/updating shared mutable data may cause race conditions and inconsistent state.

Example (race-condition fixed via synchronization):

```
class Counter {  
    private int count = 0;
```

```
    public synchronized void increment() { count++; } // synchronized ensures atomic
update
```

```
    public synchronized int getCount() { return count; }
}
```

```
public class SyncDemo {

    public static void main(String[] args) throws InterruptedException {

        Counter c = new Counter();

        Runnable r = () -> { for (int i=0;i<10000;i++) c.increment(); };

        Thread t1 = new Thread(r), t2 = new Thread(r);

        t1.start(); t2.start();

        t1.join(); t2.join();

        System.out.println("Count: " + c.getCount()); // expected 20000

    }

}
```

Q.7 c i. isAlive()

Returns true if thread is running. Example: t.isAlive().

Example:

```
class MyThread extends Thread {

    public void run() {

        try {

            System.out.println(getName() + " is running...");

            Thread.sleep(2000); // simulate some work

            System.out.println(getName() + " has finished.");

        } catch (InterruptedException e) {
```

```

        System.out.println(getName() + " interrupted.");
    }
}

}

public class IsAliveDemo {

    public static void main(String[] args) throws InterruptedException {

        MyThread t1 = new MyThread();

        t1.setName("Worker-1");

        System.out.println("Before start: isAlive = " + t1.isAlive());

        t1.start(); // start thread

        System.out.println("After start: isAlive = " + t1.isAlive());

        // Wait a little while to check status mid-way

        Thread.sleep(1000);

        System.out.println("During execution: isAlive = " + t1.isAlive());

        // Wait for thread to finish

        t1.join();

        System.out.println("After join: isAlive = " + t1.isAlive());

    }

}

```

Q.7 c ii. join()

Makes calling thread wait for another to complete. Example: t.join().

Example:

```
class Worker extends Thread {  
    private String task;  
  
    public Worker(String task) {  
        this.task = task;  
    }  
  
    public void run() {  
        for (int i = 1; i <= 3; i++) {  
            System.out.println(task + " - step " + i);  
  
            try {  
                Thread.sleep(1000); // simulate work  
            } catch (InterruptedException e) {  
                System.out.println(task + " interrupted.");  
            }  
        }  
  
        System.out.println(task + " finished.");  
    }  
}  
  
public class JoinDemo {  
    public static void main(String[] args) throws InterruptedException {  
        Worker w1 = new Worker("Task-1");  
        Worker w2 = new Worker("Task-2");  
    }  
}
```

```

w1.start();

// Main thread waits for w1 to finish before starting w2

w1.join();


w2.start();

w2.join(); // wait for w2 to finish


System.out.println("Main thread ends after both tasks complete.");
}
}

```

Q.8 a. Write a program to create multiple threads in Java.

Create Runnable tasks and start multiple threads. Example: new Thread(r).start();

Example:

```

public class MultiThreadDemo {

    public static void main(String[] args) throws InterruptedException {

        Runnable task = () -> {

            System.out.println("Running: " + Thread.currentThread().getName());

            try { Thread.sleep(200); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); }

        };

        Thread[] threads = new Thread[5];

        for (int i=0;i<threads.length;i++) {

            threads[i] = new Thread(task, "Worker-" + i);

            threads[i].start();

        }
    }
}

```

```

        for (Thread t : threads) t.join();

        System.out.println("All done.");
    }
}

```

Q.8 b. Explain the concept of thread priorities in Java. How do they affect thread execution?

Priorities range 1–10. Higher priority suggests importance, but actual scheduling is platform-dependent.

Thread priority is an integer 1–10 (MIN_PRIORITY, NORM_PRIORITY, MAX_PRIORITY). Scheduler may favor higher priority threads, but behavior is OS/JVM-dependent and not reliable for program correctness. Use coordination primitives instead of priorities for correct synchronization.

Example:

```

class MyTask extends Thread {

    public MyTask(String name) {

        super(name);
    }

    public void run() {

        for (int i = 1; i <= 5; i++) {

            System.out.println(getName() + " is running, step " + i);

        }

    }

}

public class ThreadPriorityDemo {

    public static void main(String[] args) {

```

```

MyTask t1 = new MyTask("LowPriority");

MyTask t2 = new MyTask("NormalPriority");

MyTask t3 = new MyTask("HighPriority");


// Set priorities

t1.setPriority(Thread.MIN_PRIORITY); // 1

t2.setPriority(Thread.NORM_PRIORITY); // 5 (default)

t3.setPriority(Thread.MAX_PRIORITY); // 10


// Start threads

t1.start();

t2.start();

t3.start();

}

}

```

Q.8 c. What do you understand by Byte Streams in Java? Describe their working and importance in handling binary data.

Byte Streams read/write binary data using InputStream/OutputStream. Essential for file I/O and multimedia.

Byte streams (InputStream/OutputStream and subclasses) handle raw binary data (images, audio, compressed files). Character streams (Reader/Writer) are for text (handle encoding).

Example:

```

import java.io.*;

public class CopyBinary {

    public static void main(String[] args) throws IOException {

```

```

try (InputStream in = new FileInputStream("source.bin");
    OutputStream out = new FileOutputStream("dest.bin")) {
    byte[] buf = new byte[8192];
    int len;
    while ((len = in.read(buf)) != -1) {
        out.write(buf, 0, len);
    }
}
}
}
}

```

Q.9 a. Write a Java program to handle mouse events using the `MouseListener` interface.

Implement `MouseListener` with methods like `mouseClicked()`, `mousePressed()`. Register listener on component.

`MouseListener` has 5 methods: `mouseClicked`, `mousePressed`, `mouseReleased`, `mouseEntered`, `mouseExited`. Use `MouseAdapter` if implementing only some methods.

Example:

```

import javax.swing.*;
import java.awt.event.*;

public class MouseListenerDemo extends JFrame {

    JLabel label = new JLabel("No event");

    public MouseListenerDemo() {
        super("Mouse Demo");
        setSize(400,200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}

```

```

add(label);

addMouseListener(new MouseAdapter() {

    @Override

    public void mouseClicked(MouseEvent e) {

        label.setText("Clicked at " + e.getX() + "," + e.getY());

    }

    @Override

    public void mouseEntered(MouseEvent e) { label.setText("Entered"); }

    @Override

    public void mouseExited(MouseEvent e) { label.setText("Exited"); }

});

}

public static void main(String[] args) {

    SwingUtilities.invokeLater(() -> new MouseListenerDemo().setVisible(true));

}

}

```

Q.9 b i. drawRect()

drawRect(int x, int y, int width, int height) draws the outline of a rectangle where (x,y) is the top-left

drawOval()

drawOval(int x, int y, int width, int height) draws an oval inscribed in the rectangle with the given bounds.

Example:

```
import javax.swing.*;
```

```
import java.awt.*;
```

```
public class GraphicsDemo extends JPanel {  
    protected void paintComponent(Graphics g) {  
        super.paintComponent(g);  
        g.drawRect(10,10,100,60);  
        g.drawOval(130,10,100,60);  
    }  
    public static void main(String[] args) {  
        JFrame f = new JFrame("Graphics");  
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        f.add(new GraphicsDemo());  
        f.setSize(260,120);  
        f.setVisible(true);  
    }  
}
```

Q.10 a. Explain the fundamental components of a window in a graphical user interface.

Basic GUI components:

- Top-level containers: Frame (Frame/JFrame), Dialogs.
- Containers/panels: hold components (Panel/JPanel).
- Controls/widgets: Buttons, Labels, TextFields, Checkboxes, Radio buttons, ComboBoxes, Menus.
- Layout managers: BorderLayout, FlowLayout, GridLayout, BoxLayout to arrange components.
- Event listeners: handle user input (e.g., ActionListener, MouseListener).
- Graphics context: Graphics for custom drawing.

Q.10 b. Explain how to create and display a Frame window in Java AWT. Provide a sample program to demonstrate this.

Create Frame f = new Frame("Title"); setSize(); setVisible(true);. Use addWindowListener to handle closing.

Example:

```
import java.awt.*;
```

```
import java.awt.event.*;
```

```
public class AWTFrameDemo {  
    public static void main(String[] args) {  
        Frame f = new Frame("AWT Frame Example");  
        f.setSize(400,300);  
        f.setLayout(new FlowLayout());  
        Button btn = new Button("Click Me");  
        f.add(btn);  
    }  
}
```

```
f.addWindowListener(new WindowAdapter() {  
    public void windowClosing(WindowEvent we) {  
        f.dispose(); // close window  
        System.exit(0);  
    }  
});  
  
f.setVisible(true);  
}  
}
```